

# Documento di Progetto - Web 2 (Caso A)

## Architettura e Scelte Tecnologiche

Il progetto adotta un'architettura classica basata su pattern MVC (Model-View-Controller) minimale, senza l'uso di framework enterprise come Spring o ORM complessi. Questa scelta garantisce leggerezza, velocità di avvio e pieno controllo del flusso di esecuzione, rispettando i requisiti di semplicità architetturale.

- **Maven come Build Tool:** Scelto per la gestione delle dipendenze e per standardizzare la struttura del progetto (`src/main/java`, `src/main/webapp`). Permette di compilare e pacchettizzare l'applicazione (`mvn clean package`) in un archivio `.war` in totale indipendenza dall'IDE.
- **Automazione del Deploy Locale:** In piena ottica "IDE-free", abbiamo sviluppato degli script CLI (`run.sh` e `run.bat`) che avviano automaticamente il demone MySQL, importano lo schema iniziale, compilano i sorgenti e montano l'applicativo `.war` sull'istanza Tomcat locale dell'utente, eseguendo un deploy completo in pochi secondi con un solo comando.
- **JDBC Puro e Pattern DAO:** L'interazione con il database MySQL avviene tramite chiamate JDBC standard, incapsulate nel pattern DAO. Questa scelta, unita ai Java Bean, garantisce la rigorosa separazione delle responsabilità, isolando la logica di accesso ai dati.
- **Controller a Servlet Multiple:** Il livello di controllo è frammentato su più Servlet (`MainServlet`, `CrudSimServlet`, ecc.). Ognuna intercetta le richieste del proprio dominio, valida gli input, orchestra l'interrogazione tramite i DAO e popola il contesto web. In particolare, la gestione del CRUD è centralizzata in logiche robuste di validazione per prevenire alterazioni non consentite sui contratti.
- **Thymeleaf per la Presentazione (View):** La presentazione è delegata a Thymeleaf come motore di template. Grazie alla sua sintassi naturale integrata nei tag HTML, i file (collocati in `WEB-INF/templates` per prevenirne l'accesso diretto via URL) si occupano unicamente della renderizzazione dinamica dei dati (es. cicli `for` per le tabelle), disaccoppiando completamente logica di business e presentazione.
- **Predisposizione Container (Opzionale):** Come valore aggiunto ingegneristico, il progetto è corredato da configurazioni `docker-compose` per poter essere eventualmente riprodotto in modo isolato qualora la macchina ospite non disponesse dell'ambiente locale preconfigurato.

## Manuale Utente

### Cosa si sta installando

L'applicazione "Centro Gestione Contratti" permette di gestire contratti telefonici, SIM e telefonate. L'installazione prevede il caricamento del database MySQL e la distribuzione dell'applicazione Java nel server Tomcat.

---

# 1. Deploy Locale

## Prerequisiti

1. Installare Java Development Kit (versione 8 o 21).
2. Installare Apache Tomcat (versione 9 o 11).
3. Installare MySQL Server.
4. Avviare il demone MySQL.

## Procedura di Configurazione Credenziali

*[!NOTE] Il progetto è stato testato con l'utente `root` di MySQL senza password. In questa configurazione l'installazione procede in automatico senza richiedere modifiche.*

Se MySQL richiede una password specifica, configurare le credenziali:

1. Aprire il file `run.sh` (Mac/Linux) o `run.bat` (Windows).
2. Inserire la password di MySQL nella variabile `DB_PASS`.
3. Aprire il file `src/main/java/it/unifi/progettoweb/utils/DBConnection.java`.
4. Inserire la password di MySQL nella variabile `PASSWORD`.

## Procedura di Installazione

1. Aprire un terminale.
2. Posizionarsi nella cartella principale del progetto.
3. Eseguire lo script `./run.sh` (Mac/Linux) oppure `run.bat` (Windows).
4. Attendere il completamento delle operazioni (creazione database, compilazione codice, copia su Tomcat, avvio server Tomcat).
5. Aprire un browser web.
6. Visitare l'indirizzo `http://localhost:8080/progetto-web`.

## Procedura di Arresto

1. Aprire un terminale.
2. Posizionarsi nella cartella principale del progetto.
3. Eseguire lo script `./stop.sh` (Mac/Linux) oppure `stop.bat` (Windows).

---

# 2. Deploy tramite Docker (Opzionale)

## Prerequisiti

1. Installare Docker Desktop.
2. Liberare le porte 8080 e 3306 disattivando eventuali servizi locali in ascolto.

## Procedura di Installazione

1. Avviare Docker Desktop.
2. Aprire un terminale.
3. Posizionarsi nella cartella principale del progetto.
4. Eseguire lo script `./deploy_docker.sh` (Mac/Linux) oppure `deploy_docker.bat` (Windows).
5. Attendere il completamento della procedura.
6. Aprire un browser web.
7. Visitare l'indirizzo `http://localhost:8080/progetto-web`.

## Procedura di Arresto

1. Aprire un terminale.
  2. Posizionarsi nella cartella principale del progetto.
  3. Eseguire il comando `docker-compose down`.
- 

## Risoluzione dei Problemi

### **Errore "Address already in use" all'avvio**

- Causa: La porta 8080 o 3306 risulta occupata.
- Soluzione: Individuare e terminare l'applicazione in ascolto sulla porta 8080 o 3306.

### **Permesso negato (Permission denied) su sistemi Mac/Linux**

- Causa: Mancanza di privilegi di esecuzione.
- Soluzione: Eseguire il comando `chmod +x deploy_docker.sh run.sh stop.sh mvnw`.

### **Errore "Access denied for user"**

- Causa: Credenziali del database errate.
- Soluzione: Eseguire la "Procedura di Configurazione Credenziali".

### **Errore 500 o crash durante la navigazione**

- Causa: L'applicazione Java fallisce la connessione al database MySQL.
- Soluzione: Verificare l'esecuzione del Server MySQL sulla porta 3306. Controllare le credenziali nel file `DBConnection.java`.